

Do zdobycia jest 10 punktów ale do maksymalnej punktacji z ćwiczeń wliczymy tylko 8.

- 1. (1 pkt.)** Nadleśnictwo posiada bazę danych umożliwiającą sporządzanie oraz przechowywanie tekstowych opisów poszczególnych sektorów lasu. Każdy pracownik leśny podczas obchodu lasu dysponuje aplikacją mobilną działającą w następujący sposób: w razie potrzeby edycji opisu bieżącego sektora aplikacja łączy się z bazą danych, przesyła lokalizację i pobiera krotkę dotyczącą tego sektora (w ramach pojedynczej transakcji typu READ ONLY). Pracownik może edytować pobrany opis, a po zakończeniu edycji uruchamiana jest nowa transakcja, która zapisuje wprowadzone zmiany w bazie.

Od pewnego czasu wprowadzono obowiązek aby każdy obszar leśny był regularnie wizytowany przez komisję składającą się ze specjalistów ds. ochrony przyrody, ds. gospodarki leśnej oraz innych. Specjaliści często zgłaszają, że wprowadzane przez nich modyfikacje opisów sektorów znikają mimo, że nikt nie przyznaje się do usuwania czegokolwiek. Prowadzi to do częstych nieporozumień i powstawania teorii spiskowych (np. notatka ekologa o gnieździe rzadkiego ptaka znika, a w jej miejsce pojawia się wycena drewna możliwego do pozyskania w przypadku wycinki).

Wiadomo, że system bazy danych stosuje protokół gwarantujący szeregowalność. Co w takim razie może być przyczyną problemu? W jaki sposób można się go pozbyć? Zaproponuj rozwiązanie, które pozwoli na maksymalną współbieżność (np. możesz zaproponować zmianę schematu bazy).

Uczelniana Baza Danych

Przypomnij w jaki sposób każdy z poziomów izolacji jest implementowany w PostgreSQL.

Rozważmy sytuację pewnej uczelni gdzie używana jest baza PostgreSQL. Dla każdego z następujących przypadków określ, który poziom izolacji najlepiej zastosować. Zastosuj niezależnie 2 kryteria: 1) czy może dojść do uzyskania ewidentnie niespójnych danych (np. spadek salda poniżej 0)? 2) czy wykonanie opisanych operacji zawsze będzie równoważne szeregowemu wykonaniu transakcji? Jakie będą problemy/wady jeśli zostanie wybrany inny (niższy lub wyższy) poziom niż zalecany przez Ciebie? Wy tłumacz co się dzieje powołując się na dokumentację PostgreSQL. Sprawdź eksperymentalnie (zawsze jeśli to możliwe) przed zajęciami czy Twoje rozwiązanie odpowiada rzeczywistości - uruchom odpowiednie zapytania w dwóch równoległych sesjach `psql/pgcli` i zobacz co się stanie. W czasie eksperymentowania lepiej nie implementować logiki imperatywnej (np.

IF) lecz samemu uruchamiać (lub nie) odpowiednie zapytania i polecenia COMMIT/ROLLBACK. Rozważaj wyłącznie transakcje opisane w treściach zadań, żadne inne nie będą uruchamiane w danej chwili.

Dla uzyskania punktów ważna jest rozumienie tego co się dzieje na różnych poziomach izolacji i poprawna analiza, a nie sama rekomendacja poziomu izolacji, którą w razie czego doprecyzujesz :-).

2. (1 pkt.) Aby sprawdzić poprawność danych aplikacja wylicza sumaryczną liczbę przepracowanych godzin dla wszystkich pracowników uczelni, a następnie sumaryczną liczbę przepracowanych godzin przez pracowników każdego z wydziałów. Aplikacja następnie sprawdza czy obliczone wyniki są równe. Równolegle odnotowywane są przepracowane godziny pracownikom i departamentom, każda dopisana liczba godzin w pojedynczej transakcji obejmującej operację UPDATE dla jednej i drugiej tabeli.

```
BEGIN ISOLATION LEVEL _____;  
    SELECT SUM(hours) FROM employees;  
    SELECT SUM(hours) FROM departments;  
COMMIT;
```

3. (1 pkt.) Chcemy zapewnić, że podczas przydzielania zajęć pracownikowi nie zostanie zlecone więcej godzin niż wynosi jego pensum. Zakładamy, że każdy pracownik prowadzi zajęcia na wielu kierunkach, a każdy kierunek ma swojego dyrektora dydaktycznego i wszyscy ci dyrektorzy układają plan w nocy przed deadline. Parametry zapytania ustalane przez aplikację: :newhours, :emp. W relacji plan kolumna teacher pamięta identyfikator pracownika, a hours łączną liczbę przydzielonych mu godzin. Zakładamy, że w danym momencie w bazie uruchamiane są wyłącznie transakcje takie jak poniższa.

```
BEGIN ISOLATION LEVEL _____;  
    UPDATE plan SET hours = hours + :newhours  
        WHERE teacher = :emp;  
    SELECT hours INTO asum  
        FROM plan;  
        WHERE teacher = :emp;  
    IF (asum <= 210)  
        COMMIT;  
    ELSE  
        ROLLBACK;
```

Przy okazji, jak w naturalny sposób powyższy warunek wymusić nie przejmując się poziomami izolacji?

4. (1 pkt.) Podobnie jak w poprzednim zadaniu chcemy zapewnić, że podczas przydzielania zajęć pracownikowi nie zostanie zlecone więcej godzin niż wynosi jego pensum. Parametry zapytania ustalane przez aplikację: :emp, :newhours, :course.

Tym razem inny jest schemat bazy: każda krotka w relacji **plan** ma 3 kolumny: **teacher** z identyfikatorem pracownika, **course** z identyfikatorem przedmiotu, a **hours** liczbę przydzielonych godzin z danego przedmiotu. Dodanie pracownikowi godzin polega na dopisaniu nowej krotki do tabeli **plan**. Zakładamy, że w danym momencie w bazie uruchamiane są wyłącznie transakcje takie jak poniższa.

```
BEGIN ISOLATION LEVEL _____;
  INSERT INTO plan(teacher, course, hours)
    VALUES (:emp, :course, :newhours);
  SELECT SUM(hours) INTO asum
    FROM plan;
    WHERE teacher = :emp;
  IF (asum <= 210)
    COMMIT;
  ELSE
    ROLLBACK;
```

Czy ten sam naturalny sposób zapewnieniający spełnienie warunku na maksymalne pensum może być wykorzystany również dla tego scenariusza?

5. (1 pkt.) Aplikacja przed zapisaniem studenta na termin egzamin upewnia się, że student nie zapisał się na więcej niż 2 terminy z egzaminu z podanego przedmiotu.

```
BEGIN ISOLATION LEVEL _____;
  SELECT COUNT(*) INTO anumber FROM enrol
    WHERE student = :student
      AND course = :course;
  IF (anumber <= 1)
    INSERT INTO enrol VALUES (:student, :course, :newslot);
  COMMIT
```

6. (1 pkt.) Nowy dyrektor Instytutu postanowił przyznać 10% podwyżki wszystkim pracownikom, którzy w 4-letnim okresie ewaluacyjnym opublikowali dokładnie 3 lub 4 publikacje naukowe (wiadomo, pracownik z mniej niż 3 pracami może spowodować pogorszenie wyniku ewaluacji, a z więcej niż 4 naraża społeczeństwo na niepotrzebne koszty). Spodziewamy się, że poza naszą transakcją biblioteka cały czas dopisuje kolejne publikacje (których liczba jest zapamiętywana w kolumnie **no_of_articles**).

```
BEGIN ISOLATION LEVEL _____;
  UPDATE employees
    SET remuneration = 1.10 * remuneration
    WHERE no_of_articles BETWEEN 3 AND 4;
  COMMIT
```

Wartości null

7. (1 pkt.) Mamy bazę z dwiema tabelami:

```
Bombiarze(id INT PRIMARY KEY, stopien INT),
Agenci(id INT PRIMARY KEY, bombiarz INT).
```

Bombiarze są podejrzewani o podkładanie bomb w autobusach, agenci ich śledzą, monitorują stopień zagrożenia (atrybut `stopien`) i w przypadku zebrania odpowiednich dowodów zatrzymują do wyjaśnienia. Można założyć, że jeśli bombiarz ma przydzielonego agenta to nie będzie w stanie podłożyć bomby. Agenci są dobrze zakonspirowani i czasem ich atrybut `bombiarz` ma wartość NULL - nie wiemy wtedy, którego bombiarza śledzi agent (a nawet czy w ogóle jakiegoś śledzi).

Co dokładnie wypisze poniższe zapytanie? Czy ono jest właściwe jeśli chcemy otrzymać listę bombiarzy, o których wiadomo *na pewno*, że nie mają przydzielonego agenta? A co jeśli chcemy otrzymać listę bombiarzy, którzy *być może* nie mają przydzielonego agenta?

```
SELECT id FROM Bombiarze
WHERE id NOT IN (SELECT bombiarz FROM Agenci)
```

Odpowiedzi uzasadnij. Napisz zapytania zwracające poprawne listy dla obu wariantów. Przetestuj jak szybko działają Twoje zapytania (dla bazy z milionem Bombiarzy i Agentów). W szczególności sprawdź czy lepiej użyć konstrukcji NOT IN, LEFT JOIN czy NOT EXISTS?

Indeksowanie

```
CREATE TABLE advisor(
id INTEGER PRIMARY KEY
GENERATED ALWAYS AS IDENTITY,
name text
);
```

```
CREATE TABLE student (
id INTEGER PRIMARY KEY
GENERATED ALWAYS AS IDENTITY,
advisor_id INTEGER REFERENCES advisor(id) ON DELETE RESTRICT,
name text,
last_enrol date
);
```

```
INSERT INTO advisor(name) SELECT gen_random_uuid()
FROM generate_series(1, 10000); -- tym przypiszemy potem studentów
```

```
INSERT INTO advisor(name) SELECT gen_random_uuid()
from generate_series(1, 20); -- krotki bez przypisanego studenta
```

```
INSERT INTO student(advisor_id, name, last_enrol)
SELECT floor(1 + random() * 10000)::int,
gen_random_uuid(),
date '2020-01-01' +
    random() * (interval '5 years')
FROM generate_series(1, 2000000);
```

8. (1 pkt.) Rozważmy operacje znajdowania oraz operację usuwania wszystkich krotek z tabeli `advisor`, które nie mają przypisanego studenta. Przeprowadź eksperymenty i omów ich wyniki. Użyj `EXPLAIN ANALYZE`. Dlaczego pierwsza z nich trwa szybciej niż druga?

W szczególności dowiedz się jakiej operacji dotyczy linijka:

```
Trigger for constraint student_advisor_id_fkey: time=1932.326 calls=20
```

Zaproponuj indeks, który przyspieszy operację usuwania. Dlaczego taki indeks nie przyspiesza głównej operacji wybierania krotek z `advisor` bez przypisanego studenta.

9. (1 pkt.) Rozważamy następującą tabelę:

```
registration( -- zapis do grupy
    id uuid PRIMARY KEY,
    student_id int REFERENCES student(id),
    group_id int REFERENCES agroup(id),
    tmstp timestamp NOT NULL,
    deleted bool NOT NULL);
```

System działa w taki sposób, że po każdym zapisie studenta na zajęcia wykonywana jest operacja `INSERT` z kolumną `id` o wartości wygenerowanej przez `gen_random_uuid()` oraz `tmstp` ustawioną na `now()`. Wszystkie pozostałe operacje na tej tabeli jedynie czytają dane lub modyfikują flagę `deleted`. Rozważmy dwa zapytania:

```
SELECT student_id, tmstp FROM registration ORDER BY student_id;
SELECT student_id, tmstp FROM registration ORDER BY student_id LIMIT 10;
```

W celu ich przyspieszenia założono dodatkowy indeks (`btree`) na kolumnie `student_id` oraz wydano polecenie `ANALYZE`. W przeprowadzonym eksperymencie drugie zapytanie przyspieszyło znacząco, ale pierwsze nie tylko nie przyspieszyło, ale nawet minimalnie spowolniło. `EXPLAIN ANALYZE` pokazuje, że oba zapytania są wykonywane z wykorzystaniem nowego indeksu. Wyłumacz dlaczego tak się stało.

W jaki sposób można przyspieszyć działanie pierwszego z powyższych zapytań, gdyby było to priorytetowe dla Twojego systemu? Dlaczego zaproponowany przez Ciebie sposób pomaga? Jakie ma wady?

Na wykładzie pojawiły się dwa takie sposoby.

10. (1 pkt.) Rozważmy tabele:

```
registration(  -- zapis do grupy
  student_id int REFERENCES student(id),
  agroup_id int REFERENCES agroup(id),
  tmstp timestamp NOT NULL,
  PRIMARY KEY(student_id, agroup_id));

queue(  -- zapis do kolejki oczekujących
  student_id int REFERENCES student(id),
  agroup_id int REFERENCES agroup(id),
  tmstp timestamp NOT NULL,
  PRIMARY KEY(student_id, agroup_id));

agroup(  -- informacje o grupie
  id int PRIMARY KEY,
  max_size int NOT NULL,
  teacher_id int);
```

W systemie zapisów działa trigger typu **FOR EACH ROW, BEFORE INSERT**, który przy każdej próbie zapisania studenta do grupy (czyli **INSERT** do **registration**) sprawdza czy została osiągnięta jej maksymalna pojemność i jeśli tak to zapisuje krotkę **NEW** do tabeli **queue** oraz blokuje zapis do **registration** zwracając wartość **NULL**.

Dodatkowo działa trigger typu **FOR EACH ROW, AFTER DELETE**, który po usunięciu krotki z tabeli **registration** znajduje krotkę z **queue** dotyczącą danej grupy, która ma najwcześniejszy **tmstp** i jeśli taka krotka istnieje (powiedzmy, że jej klucz to **:s_id, :g_id**), to usuwa ją z **queue** za pomocą operacji **DELETE FROM queue WHERE student_id = :s_id AND agroup_id = :g_id**, a następnie dodaje do **registration**.

Wszystkie transakcje działają na poziomie **READ COMMITTED**. Nie są uruchamiane żadne polecenia **UPDATE**. Wszystkie operacje **INSERT** są wywoływane z **tmstp** ustawionym jako wartość **now()**.

1. Czy możliwy jest scenariusz, w którym dochodzi do przekroczenia pojemności jakiejś grupy? Uzasadnij odpowiedź, a jeśli jest ona pozytywna to podaj przykład.
2. Zauważyłeś, że w czasie gdy studenci jedynie wypisują się z grup, czasami pojawia się błąd
ERROR: duplicate key value violates unique constraint. Jak to możliwe? Skonstruuj odpowiedni przykład.

3. Czy podane przez Ciebie scenariusze pozostaną możliwe, jeśli wszystkie transakcje będą działały na poziomie REPEATABLE READ?